

Tarefa 4: Programas Memory-Bound vs. Compute-Bound com OpenMP

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Junho de 2026

1. Introdução

Este relatório apresenta a análise experimental do desempenho de dois programas paralelos implementados em linguagem C com a API OpenMP: um limitado pela largura de banda de memória (memory-bound) e outro limitado pela capacidade de processamento da unidade de ponto flutuante (compute-bound). O objetivo é observar empiricamente como cada classe de programa responde ao aumento do número de threads, identificando os gargalos que impedem a escalabilidade ideal.

A paralelização foi realizada com a diretiva `#pragma omp parallel for`, variando-se o número de threads entre 1, 2, 4, 8 e 16, em um processador com 4 núcleos físicos e suporte a Simultaneous Multithreading (SMT), resultando em 8 threads lógicas.

2. Ambiente de Execução

A Tabela 1 descreve o ambiente de hardware e software utilizado nos experimentos.

Tabela 1 — Configuração do ambiente experimental

Item	Valor
CPU	AMD Ryzen 5 5600G
Núcleos físicos / Threads lógicas	4 núcleos / 8 threads (SMT)
Compilador	GCC com -O2 -fopenmp
Sistema Operacional	Linux (WSL2)

3. Implementação

3.1 Programa Memory-Bound

O programa `memory_bound.c` realiza a soma de redução de um vetor de 100 milhões de elementos do tipo `double`, totalizando aproximadamente 800 MB de dados. Esse tamanho é deliberadamente superior à cache L3 do processador, garantindo que cada acesso ao

vetor resulte em uma busca à memória principal. A operação por elemento é trivial, tornando o gargalo exclusivo a largura de banda do barramento de memória:

```
#pragma omp parallel for reduction(+:soma) schedule(static)
for (long i = 0; i < N; i++)
    soma += v[i];
```

3.2 Programa Compute-Bound

O programa `compute_bound.c` aplica 80 operações de ponto flutuante pesadas por elemento — envolvendo `sin`, `cos`, `log`, `exp` e `sqrt` encadeados em 20 rodadas — sobre um vetor de 10 milhões de elementos. Os dados cabem inteiramente na cache, fazendo com que o gargalo seja exclusivamente a unidade de ponto flutuante (FPU) do processador:

```
for (int j = 0; j < ROUNDS; j++) {
    r = sin(r) + cos(r * 1.1);
    r = log(fabs(r) + 1.0) + exp(-fabs(r) * 0.01);
    r = sqrt(fabs(r) + 1e-9);
}
```

4. Resultados

4.1 Memory-Bound

A Tabela 2 apresenta os resultados de tempo e speedup para o programa `memory-bound`.

Tabela 2 — Resultados: `memory_bound.c`

Threads	Tempo (s)	Speedup	Observação
1	0,0709	1,00	Baseline
2	0,0372	1,91	Boa escalabilidade
4	0,0199	3,56	Máximo observado
8	0,0226	3,14	Queda de desempenho
16	0,0229	3,10	Sem ganho adicional

4.2 Compute-Bound

A Tabela 3 apresenta os resultados de tempo e speedup para o programa `compute-bound`.

Tabela 3 — Resultados: `compute_bound.c`

Threads	Tempo (s)	Speedup	Observação
1	7,0275	1,00	Baseline
2	3,5750	1,97	Escalabilidade quase linear

4	1,8604	3,78	Máximo observado
8	1,8434	3,81	Platô (SMT sem ganho)
16	1,8384	3,82	Sem ganho adicional

5. Análise e Discussão

5.1 Memory-Bound: saturação antes dos núcleos físicos

O speedup cresce de forma expressiva entre 1 e 4 threads ($1,00\times \rightarrow 3,56\times$), mas sofre queda ao avançar para 8 threads ($3,14\times$) e se mantém estagnado com 16. O fenômeno ocorre porque a largura de banda do barramento de memória é um recurso compartilhado e finito. Ao saturá-lo com 4 threads, adicionar mais threads intensifica a disputa pelo mesmo canal sem elevar a taxa de transferência efetiva.

O ganho observado até 4 threads reflete, em parte, a capacidade do controlador de memória de realizar prefetching de forma mais eficaz quando múltiplas threads emitem requisições contíguas. Contudo, esse benefício se esgota assim que o barramento atinge sua capacidade máxima.

No que diz respeito ao SMT, o hyperthreading pode, em tese, encobrir latências de acesso à memória alternando a execução entre threads enquanto uma aguarda dados. Porém, quando o barramento já está saturado, esse mecanismo perde eficácia, resultando na queda de speedup observada.

5.2 Compute-Bound: hyperthreading sem ganho

O speedup escala de forma próxima ao ideal entre 1 e 4 threads ($1,00\times \rightarrow 3,78\times$), refletindo a distribuição eficiente do trabalho entre os 4 núcleos físicos. A partir de 8 threads, o ganho torna-se desprezível ($3,81\times$ com 8, $3,82\times$ com 16).

Esse comportamento evidencia a **limitação fundamental do hyperthreading para cargas compute-bound**: dois threads lógicos compartilham a mesma FPU no mesmo núcleo físico. Ao saturar a unidade de ponto flutuante com o primeiro thread lógico, o segundo aguarda o mesmo recurso sem executar cálculos úteis. O SMT foi concebido para esconder latências de memória, não para duplicar a capacidade de cálculo.

5.3 Resumo das limitações

A Tabela 4 resume as causas e efeitos observados em cada cenário.

Tabela 4 — Resumo das limitações de escalabilidade

Situação	Causa	Efeito
Memory-bound com > 4 threads	Saturação da largura de banda de memória	Speedup estabiliza ou cai

Compute-bound com SMT (> 4 threads)	FPU compartilhada entre threads lógicos	Hyperthreading sem ganho real
Memory-bound com SMT	Latência de memória parcialmente encoberta	Melhora leve até saturar barramento

6. Conclusão

Os experimentos confirmaram que a natureza do gargalo determina os limites práticos da paralelização com OpenMP. Programas memory-bound atingem sua capacidade máxima quando a largura de banda do barramento é saturada, independentemente do número de threads ou núcleos disponíveis. Programas compute-bound escalam bem até o número de núcleos físicos, mas o SMT não oferece ganhos reais quando a FPU é o recurso limitante.

Esses resultados têm implicações diretas no projeto de aplicações de alto desempenho: conhecer o gargalo da aplicação é pré-requisito para a escolha adequada de estratégias de paralelização e para a definição realista de metas de speedup.

7. Código Fonte

7.1 Memory-bound

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
5  #include <omp.h>
6
7  #define N      100000000L /* 100M doubles = ~800 MB, não cabe no cache */
8  #define BILLION 1000000000.0
9
10 static double elapsed(struct timespec a, struct timespec b) {
11     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
12 }
13
14 int main(void) {
15     double *v = malloc(N * sizeof(double));
16     if (!v) { fprintf(stderr, "malloc falhou\n"); return 1; }
17
18     for (long i = 0; i < N; i++) v[i] = (double)i * 0.5;
19
20     int threads[] = {1, 2, 4, 8, 16};
21     int n_casos   = sizeof(threads) / sizeof(threads[0]);
22
23     printf("Memory-bound: soma de vetor com %ldM elementos\n\n", N / 1000000L);
24     printf("%-8s  %-14s  %-10s  %-10s\n",
25           "Threads", "Soma", "Tempo(s)", "Speedup");
26     printf("%-8s  %-14s  %-10s  %-10s\n",
27           "-----", "-----", "-----", "-----");
28
29     double t_base = 0.0;
30     for (int k = 0; k < n_casos; k++) {
31         int t = threads[k];
32         omp_set_num_threads(t);
33
34         struct timespec ini, fim;
35         double soma = 0.0;
36
37         clock_gettime(CLOCK_MONOTONIC, &ini);
38         #pragma omp parallel for reduction(+:soma) schedule(static)
39         for (long i = 0; i < N; i++)
40             soma += v[i];
41         clock_gettime(CLOCK_MONOTONIC, &fim);
42
43         double tempo = elapsed(ini, fim);
44         if (k == 0) t_base = tempo;
45
46         printf("%-8d  %-14.2e  %-10.4f  %-10.4f\n",
47               t, soma, tempo, t_base / tempo);
48     }
49
50     free(v);
51     return 0;
52 }
```

7.2 Compute-bound

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #include <time.h>
6  #include <omp.h>
7
8  /* Cada iteração executa várias operações de ponto flutuante pesadas,
9     tornando o gargalo o hardware de FPU, não a memória. */
10 #define N      10000000L /* 10M elementos – dados cabem no cache */
11 #define ROUNDS 20        /* repetições de sin/cos/log/exp por elemento */
12 #define BILLION 100000000.0
13
14 static double elapsed(struct timespec a, struct timespec b) {
15     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
16 }
17
18 static double calcular(double x) {
19     double r = x;
20     for (int j = 0; j < ROUNDS; j++) {
21         r = sin(r) + cos(r * 1.1);
22         r = log(fabs(r) + 1.0) + exp(-fabs(r) * 0.01);
23         r = sqrt(fabs(r) + 1e-9);
24     }
25     return r;
26 }
27
28 int main(void) {
29     double *v = malloc(N * sizeof(double));
30     double *u = malloc(N * sizeof(double));
31     if (!v || !u) { fprintf(stderr, "malloc falhou\n"); return 1; }
32
33     for (long i = 0; i < N; i++) v[i] = (double)(i + 1);
34
35     int threads[] = {1, 2, 4, 8, 16};
36     int n_casos = sizeof(threads) / sizeof(threads[0]);
37
38     printf("Compute-bound: %d operações FP pesadas por elemento, %ldM elementos\n\n",
39           ROUNDS * 4, N / 1000000L);
40     printf("%-8s  %-14s  %-10s  %-10s\n",
41           "Threads", "Checksum", "Tempo(s)", "Speedup");
42     printf("%-8s  %-14s  %-10s  %-10s\n",
43           "-----", "-----", "-----", "-----");
44
45     double t_base = 0.0;
46     for (int k = 0; k < n_casos; k++) {
47         int t = threads[k];
48         omp_set_num_threads(t);
49
50         struct timespec ini, fim;
51         double checksum = 0.0;
52
53         clock_gettime(CLOCK_MONOTONIC, &ini);
54         #pragma omp parallel for reduction(+:checksum) schedule(static)
55         for (long i = 0; i < N; i++) {
56             u[i] = calcular(v[i]);
57             checksum += u[i];
58         }
59         clock_gettime(CLOCK_MONOTONIC, &fim);
60
61         double tempo = elapsed(ini, fim);
62         if (k == 0) t_base = tempo;
63
64         printf("%-8d  %-14.6f  %-10.4f  %-10.4f\n",
65               t, checksum, tempo, t_base / tempo);
66     }
67
68     free(v);
69     free(u);
70     return 0;
71 }
```